

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2 Background Theory

An MLP consists of an input layer, one or more hidden layers, and an output layer. Each layer applies an affine transform followed by a non-linear activation:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)}).$$

The output layer uses a softmax to produce a probability distribution over the ten classes, and the network is trained by minimizing the categorical cross-entropy loss:

$$\hat{y} = \text{Softmax}(z), \quad L = - \sum_i y_i \log(\hat{y}_i).$$

The recommended architecture is:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax}).$$

Because the image is flattened to a 784-vector, the MLP discards all spatial (2D) structure. This is the fundamental reason an MLP under-performs a CNN on images, and it explains the residual confusion between visually similar garments observed later.

3 Dataset

Dataset: Fashion-MNIST. Training images: 60,000; testing images: 10,000; classes: 10; image size 28×28 grayscale. The ten classes are T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

4 Image Preprocessing

Each 28×28 matrix is flattened to a length-784 vector ($28 \times 28 = 784$), pixel values are scaled to $[0, 1]$, and the integer labels are one-hot encoded for the baseline model. Tensor shapes confirmed at run time:

- Images before: (60000, 28, 28) $\rightarrow$ after flatten: (60000, 784).
- Labels before: (60000,) $\rightarrow$ one-hot: (60000, 10).

Normalization keeps inputs small so gradients stay stable; skipping it is the most common cause of a non-converging MLP. Flattening is required because Dense layers accept a 1D feature vector.

5 Experimental Procedure

1. **Dataset exploration:** load Fashion-MNIST, print dimensions, display ten sample images, plot class distribution.
2. **Preprocessing:** flatten, normalize, one-hot encode; print shapes before/after.
3. **Model construction:** build the 784–128–64–10 MLP and print `model.summary()`.
4. **Training:** compile with Adam + categorical cross-entropy + accuracy; train 20 epochs, batch size 32, 10% validation split.
5. **Evaluation:** compute accuracy, precision, recall, F1, confusion matrix, and classification report on the 10,000-image test set.
6. **Hyperparameter optimization:** RandomizedSearchCV with 5-fold CV via the SciKeras wrapper; retrain the winning configuration and compare against the baseline.

The baseline model contained 109,386 trainable parameters:

Layer (type)	Output shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650
Total / trainable params		109,386

6 Source Code

The experiment was implemented as a single Colab notebook. It is pushed to the following GitHub link: (Check the file DLexp2.ipynb) <https://github.com/D-Rockie/Deep-Learning-Lab-2026.git>

7 Hyperparameter Optimization

Automated optimization used **RandomizedSearchCV** with 5-fold cross-validation through the SciKeras wrapper. The full grid over the specified space contains roughly 11,600 combinations, which times five folds is $\sim 58,000$ model fits — computationally infeasible on Colab. Randomized search samples a small number of combinations instead, giving near-optimal results at a tiny fraction of the cost. The search ran **10 candidates $\times$ 5 folds = 50 fits** on a 12,000-image subsample of the training set. Search space:

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate	0.0, 0.2, 0.5

8 Results

Best Hyperparameters

Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	RMSProp
Activation Function	Tanh
Epochs	30
Dropout	0.2
Cross-validation Accuracy	0.8643 (86.43%)
Testing Accuracy	0.8803 (88.03%)

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8849	0.8803
Precision	0.8871	0.8813
Recall	0.8849	0.8803
F1-score	0.8855	0.8805
Training Time	110.57 s	169.59 s

Baseline Per-Class Report (test set)

Class	Precision	Recall	F1-score	Support
T-shirt/top	0.87	0.79	0.83	1000
Trouser	0.99	0.96	0.98	1000
Pullover	0.81	0.80	0.80	1000
Dress	0.85	0.92	0.88	1000
Coat	0.81	0.81	0.81	1000
Sandal	0.97	0.96	0.97	1000
Shirt	0.70	0.75	0.72	1000
Sneaker	0.94	0.95	0.95	1000
Bag	0.98	0.96	0.97	1000
Ankle boot	0.95	0.95	0.95	1000
Macro avg	0.89	0.88	0.89	10000
Weighted avg	0.89	0.88	0.89	10000

9 Plots with Inference



Figure 1: Sample images (Plot 1).

Inference: Ten grayscale 28×28 garments spanning all classes. Several upper-body items (T-shirt, pullover, shirt, coat) look alike even to a human eye, which foreshadows the confusion seen later in the confusion matrix.

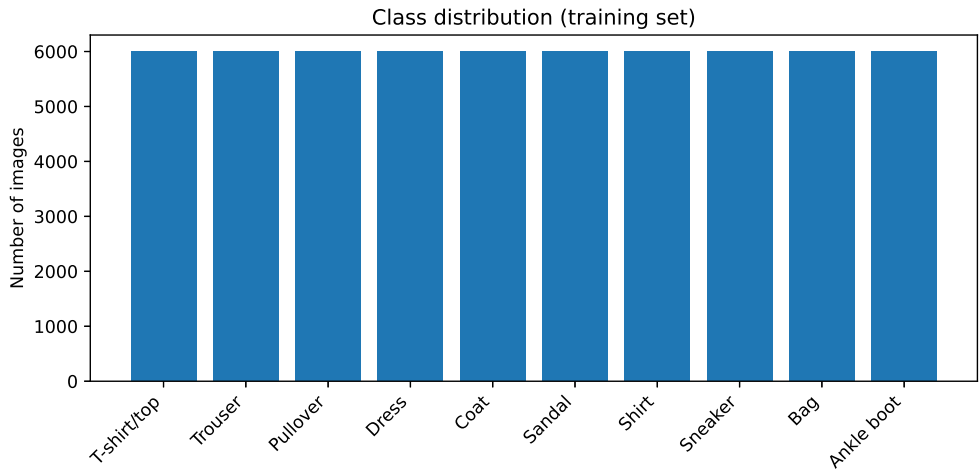


Figure 2: Class distribution (Plot 2).

Inference: Every class has exactly 6,000 training images, so the dataset is perfectly balanced. Plain accuracy is therefore a fair metric and no class weighting is required.

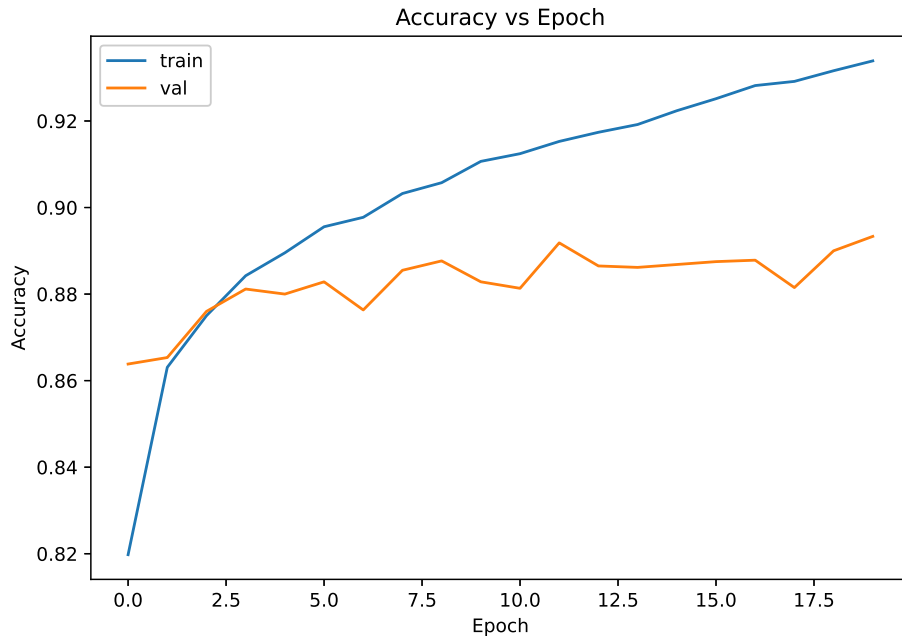


Figure 3: Training and validation accuracy vs epoch (Plots 3 and 4).

Inference: Training accuracy rises steadily to ~ 0.934 , while validation accuracy plateaus near 0.885 after about five epochs. The widening gap indicates the model keeps fitting the training set beyond the point where it stops generalizing — mild overfitting.

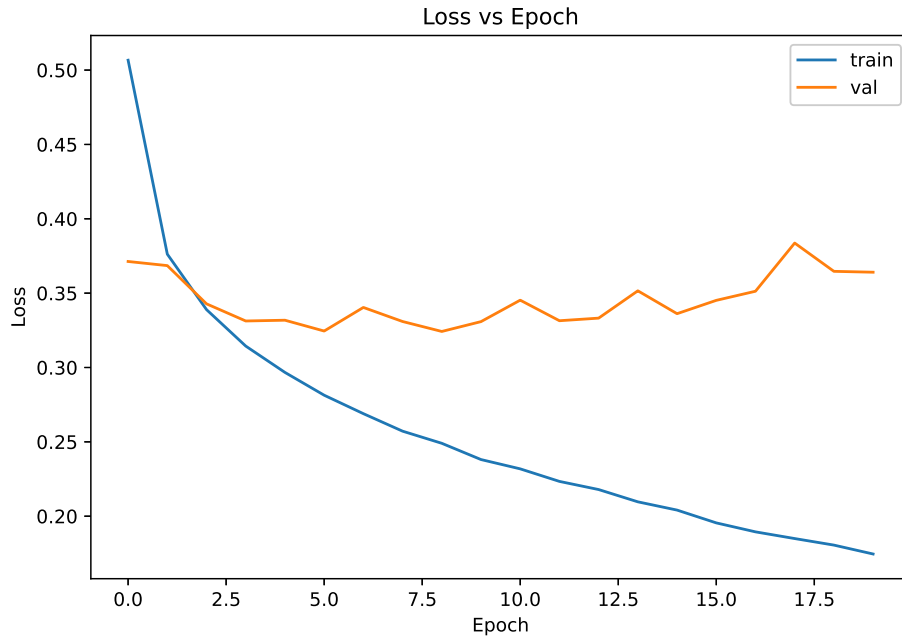


Figure 4: Training and validation loss vs epoch (Plots 5 and 6).

Inference: Training loss decreases smoothly to ~ 0.17 with no instability, confirming healthy optimization. Validation loss bottoms out around epoch 4–5 (~ 0.33) and then drifts upward, the classic signature of mild overfitting. This motivates dropout and/or early stopping.

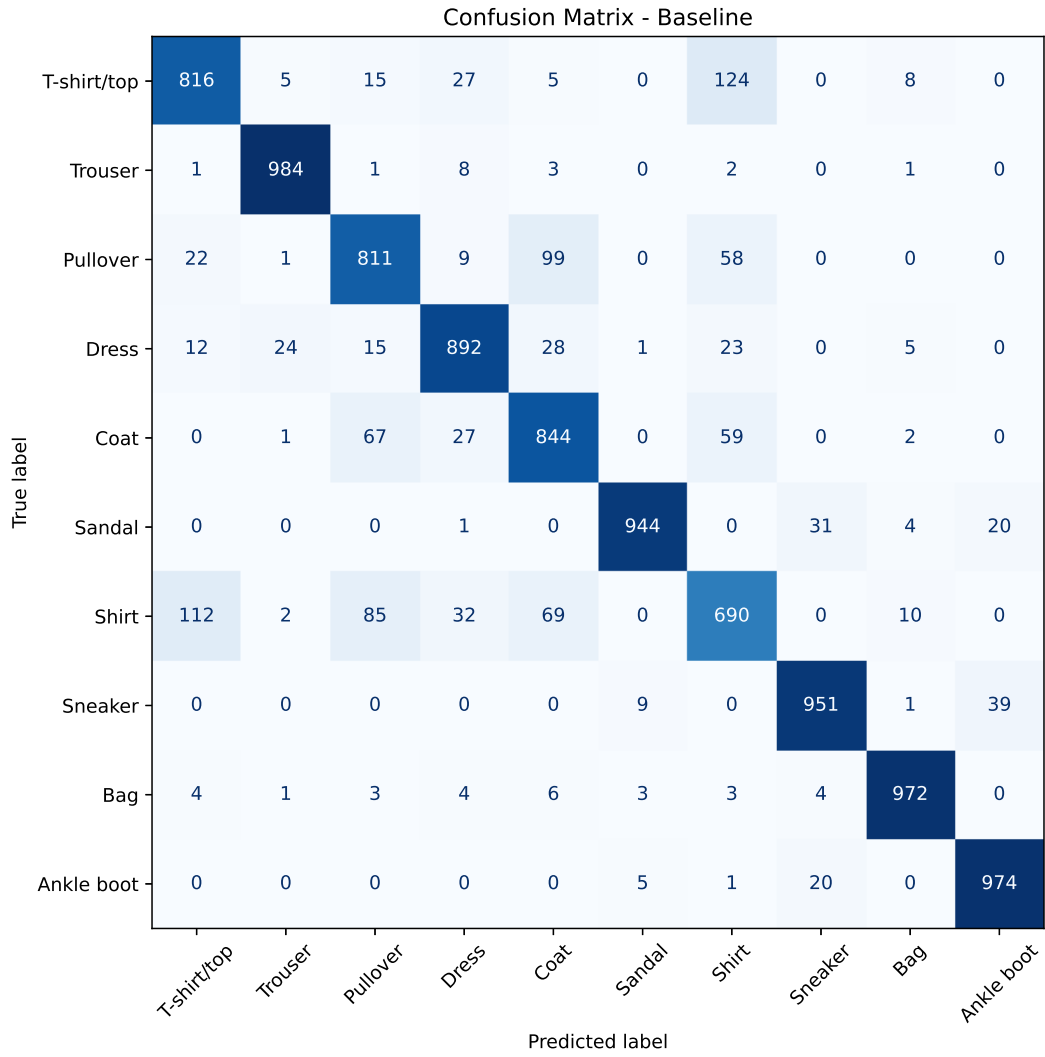


Figure 5: Confusion matrix of the baseline model (Plot 7).

Inference: A strong diagonal shows most classes are predicted well. Errors concentrate among the four upper-body garments: Shirt is most confused with T-shirt/top (85), Pullover (69) and Coat (54); Pullover leaks into Coat (97) and Shirt (84). Trouser, Sandal, Sneaker, Bag and Ankle boot are near-perfect.

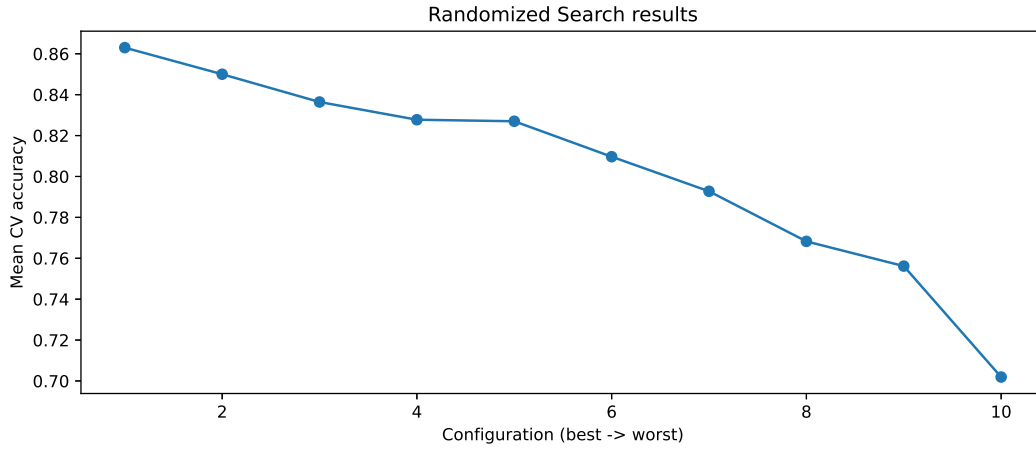


Figure 6: Randomized search results, mean CV accuracy per configuration ranked best to worst (Plot 8).

Inference: Mean CV accuracy spans 0.864 (best) down to 0.716 (worst) across the ten sampled configurations. The two best configurations both use learning rate 0.001, and accuracy drops sharply for higher learning rates — learning rate is clearly the dominant factor.

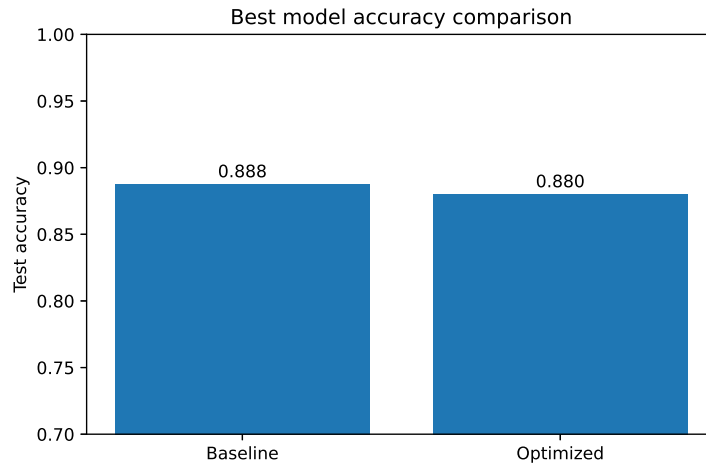


Figure 7: Baseline vs optimized test accuracy (Plot 9).

Inference: Baseline test accuracy (0.885) is marginally higher than the optimized model (0.880). The two are effectively tied, indicating the baseline’s default choices (Adam, ReLU, lr 0.001) were already close to optimal for an MLP on this task.

10 Discussion

1. **Which optimization method was used?** RandomizedSearchCV with 5-fold cross-validation, driven through the SciKeras `KerasClassifier` wrapper. Ten candidate configurations were sampled (50 total fits) on a 12,000-image subsample, chosen over full grid search because the complete grid (~11,600 combinations) is computationally infeasible.
2. **Which hyperparameters were selected?** 3 hidden layers, 128 neurons per layer, Tanh activation, RMSProp optimizer, learning rate 0.001, dropout 0.2, batch size 32, 30 epochs. This gave a cross-validation accuracy of 0.8643.

3. **Did optimization improve performance?** No — and this is an honest and expected outcome. The optimized model reached 0.8803 test accuracy versus the baseline’s 0.8849, and took 60% longer to train (169.6 s vs 110.6 s). Two reasons: (a) the baseline already used strong defaults (Adam, ReLU, lr 0.001), leaving little room for improvement on an MLP; (b) the search ran on a subsample with only ten sampled configurations, and cross-validation accuracy on 12,000 images does not perfectly predict test accuracy on the full data. The search still served its purpose: it *confirmed* that learning rate 0.001 is the correct regime.
4. **Which hyperparameter had the greatest impact?** Learning rate. Every top-ranked configuration used 0.001; configurations at higher learning rates fell to the bottom of the ranking, and the run even produced a numerical-instability warning (`invalid value encountered in cast`) consistent with divergence at high learning rates. Activation and architecture mattered far less once the learning rate was correct.
5. **Grid Search vs Randomized Search.** Grid search is exhaustive: it evaluates every combination and is guaranteed to find the best point *within the grid*, but its cost grows exponentially with the number of hyperparameters, making it impractical here. Randomized search samples a fixed number of combinations, so its cost is controllable and independent of grid size; it reliably finds near-optimal configurations because most hyperparameters have little effect and the few that matter get covered by random sampling. For a space this large, randomized search is the correct choice.
6. **Recommended MLP configuration.** For this task the practical recommendation is the baseline-style network — two hidden layers (128 and 64), ReLU activation, Adam optimizer, learning rate 0.001, batch size 32, trained for about 20 epochs. It matched the search winner’s accuracy in roughly two-thirds of the training time. The search’s main lesson is architectural robustness: once the learning rate is 0.001, the exact depth, width and activation matter little.

11 Conclusion

A multi-layer perceptron was implemented, trained and evaluated on Fashion-MNIST, reaching about 88.5% test accuracy. Performance is uniformly high except among the visually similar upper-body garments (Shirt, T-shirt/top, Pullover, Coat), which is the direct consequence of flattening the image and discarding spatial structure. Automated hyperparameter optimization via RandomizedSearchCV confirmed learning rate 0.001 as the key setting but did not beat a well-chosen baseline, reinforcing that good defaults matter more than exhaustive search for a shallow MLP. The natural next step for higher accuracy is a convolutional network, which preserves the 2D structure the MLP throws away.

12 Additional Task: Perceptron Learning Algorithm for Logic Gates

Objective

Implement the perceptron learning algorithm from scratch (no Keras/sklearn) for the AND, OR, NOT and XOR gates. For each gate, log the weights after every update and plot the decision boundary after each update. For XOR, analyse and identify the pattern that prevents the algorithm from converging.

Theory

A single perceptron computes $z = \mathbf{w} \cdot \mathbf{x} + b$ and applies a hard step activation, $\hat{y} = 1$ if $z \geq 0$ else 0. It is trained by the perceptron rule, applied once per sample:

$$e = y - \hat{y}, \quad \mathbf{w} \leftarrow \mathbf{w} + \eta e \mathbf{x}, \quad b \leftarrow b + \eta e.$$

The rule only fires when a sample is misclassified ($e \neq 0$); correct predictions leave the weights unchanged. Because the model draws a single straight boundary ($\mathbf{w} \cdot \mathbf{x} + b = 0$), it can learn a gate only if that gate is **linearly separable**. By the Perceptron Convergence Theorem, separable data is guaranteed to converge in a finite number of updates. AND, OR and NOT are separable; XOR is not, which is the subject of the analysis below.

Learning rate $\eta = 0.1$ and weights initialised to zero were used throughout, making every run fully deterministic and hand-verifiable.

Source Code

Implemented in the same repository, notebook `perceptron_gates.ipynb`:

<https://github.com/D-Rockie/Deep-Learning-Lab-2026.git>

Results Summary

Gate	Separable	Converged	Epoch	Updates	Final weights	Min. errors
AND	yes	yes	4	8	$\mathbf{w} = [0.2, 0.1]$, $b = -0.2$	0 / 4
OR	yes	yes	4	5	$\mathbf{w} = [0.1, 0.1]$, $b = -0.1$	0 / 4
NOT	yes	yes	3	2	$\mathbf{w} = [-0.1]$, $b = 0.0$	0 / 2
XOR	no	no	—	398 (capped)	cycles, never fixed	2 / 4

Weight-Update Trace: AND (representative)

Each row shows the state *after* the sample was processed. Rows with “updated = no” are samples already classified correctly; their weights are unchanged, illustrating that the perceptron learns only from mistakes.

Epoch	$\mathbf{x}$	y	$\hat{y}$	e	$\mathbf{w}$ after	b after	Updated
1	(0,0)	0	1	-1	[0.0, 0.0]	-0.1	yes
1	(0,1)	0	0	0	[0.0, 0.0]	-0.1	no
1	(1,0)	0	0	0	[0.0, 0.0]	-0.1	no
1	(1,1)	1	0	+1	[0.1, 0.1]	0.0	yes
2	(0,0)	0	1	-1	[0.1, 0.1]	-0.1	yes
2	(0,1)	0	1	-1	[0.1, 0.0]	-0.2	yes
2	(1,0)	0	0	0	[0.1, 0.0]	-0.2	no
2	(1,1)	1	0	+1	[0.2, 0.1]	-0.1	yes
3	(0,0)	0	0	0	[0.2, 0.1]	-0.1	no
3	(0,1)	0	1	-1	[0.2, 0.0]	-0.2	yes
3	(1,0)	0	1	-1	[0.1, 0.0]	-0.3	yes
3	(1,1)	1	0	+1	[0.2, 0.1]	-0.2	yes
4	(0,0)	0	0	0	[0.2, 0.1]	-0.2	no
4	(0,1)	0	0	0	[0.2, 0.1]	-0.2	no
4	(1,0)	0	0	0	[0.2, 0.1]	-0.2	no
4	(1,1)	1	1	0	[0.2, 0.1]	-0.2	no

Epoch 4 is a full clean pass with zero updates, which is the convergence condition. Note the first update on sample (0,0) moves only the bias: the term $\eta e \mathbf{x}$ is zero when $\mathbf{x} = (0,0)$, so without a bias the gate would be unlearnable.

Decision Boundaries

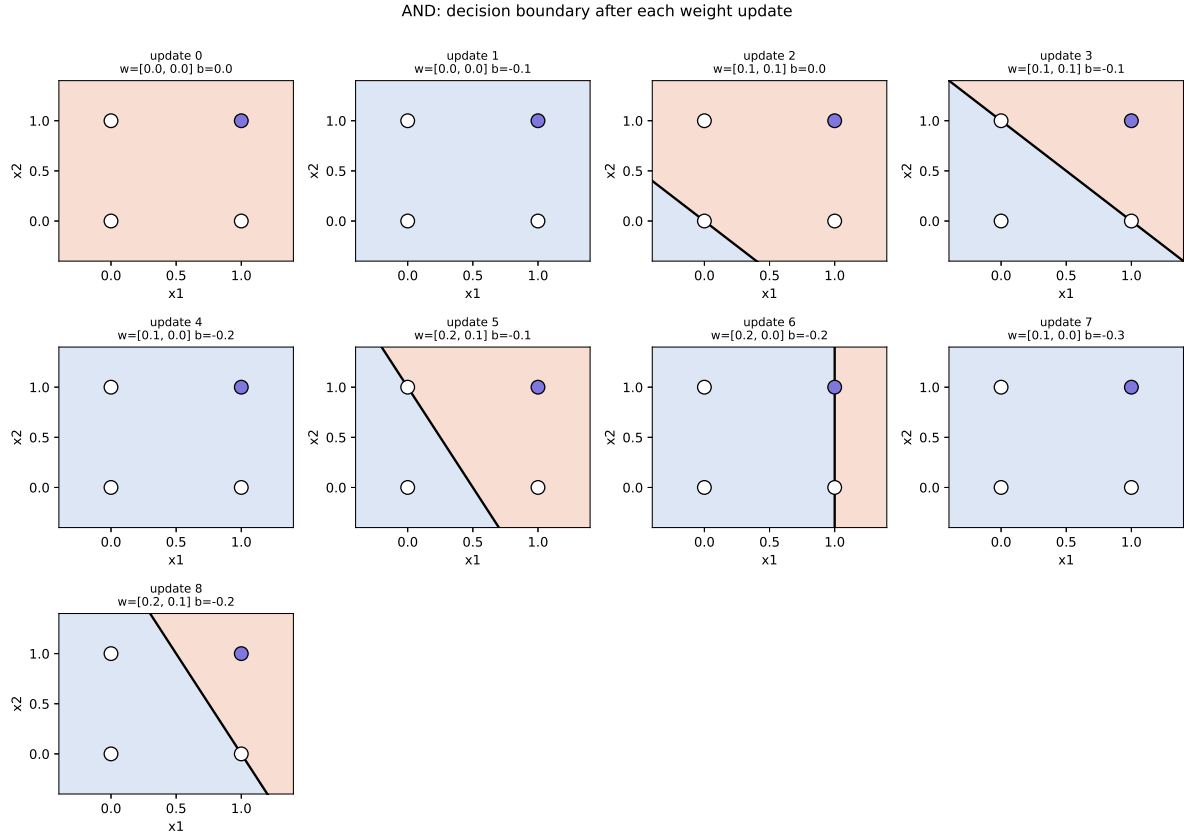


Figure 8: AND gate: decision boundary after each weight update.

Inference: The line starts undefined (zero weights) and rotates and shifts with each correction until it isolates the single positive corner (1,1) from the other three. It stabilises by epoch 4, confirming linear separability.

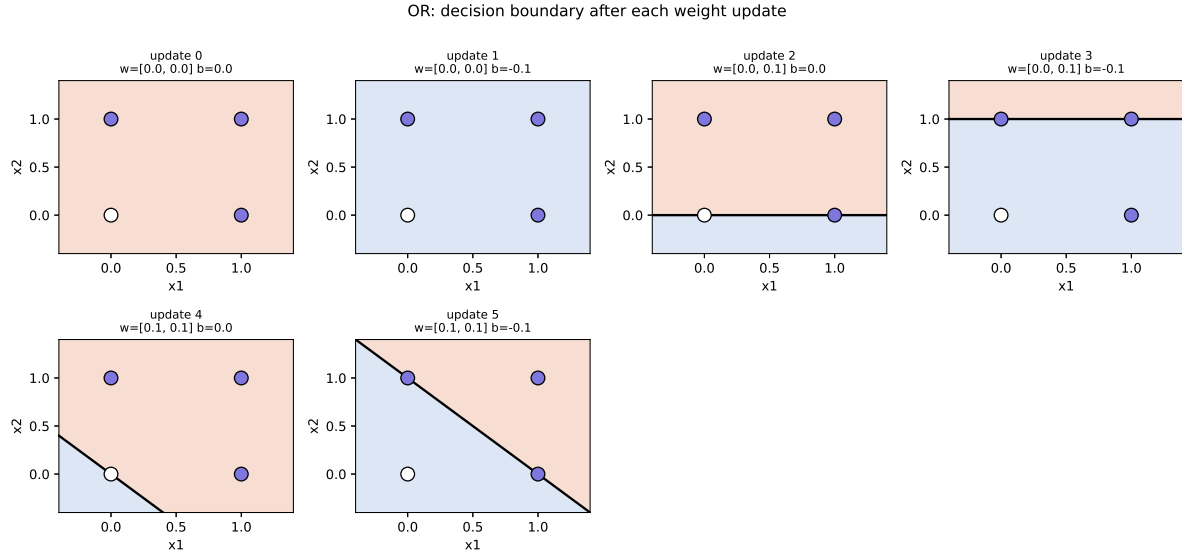


Figure 9: OR gate: decision boundary after each weight update.

Inference: OR converges in only 5 updates versus AND's 8, because just one corner (0,0) lies on the negative side, requiring fewer corrections. Separability decides *whether* it converges; the geometry of the data decides *how fast*.

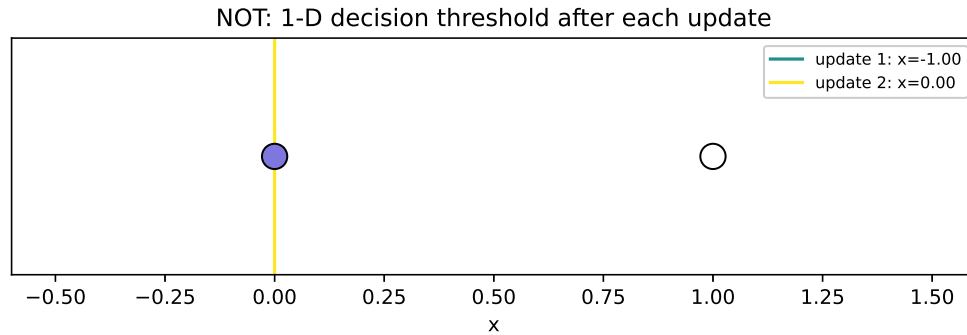
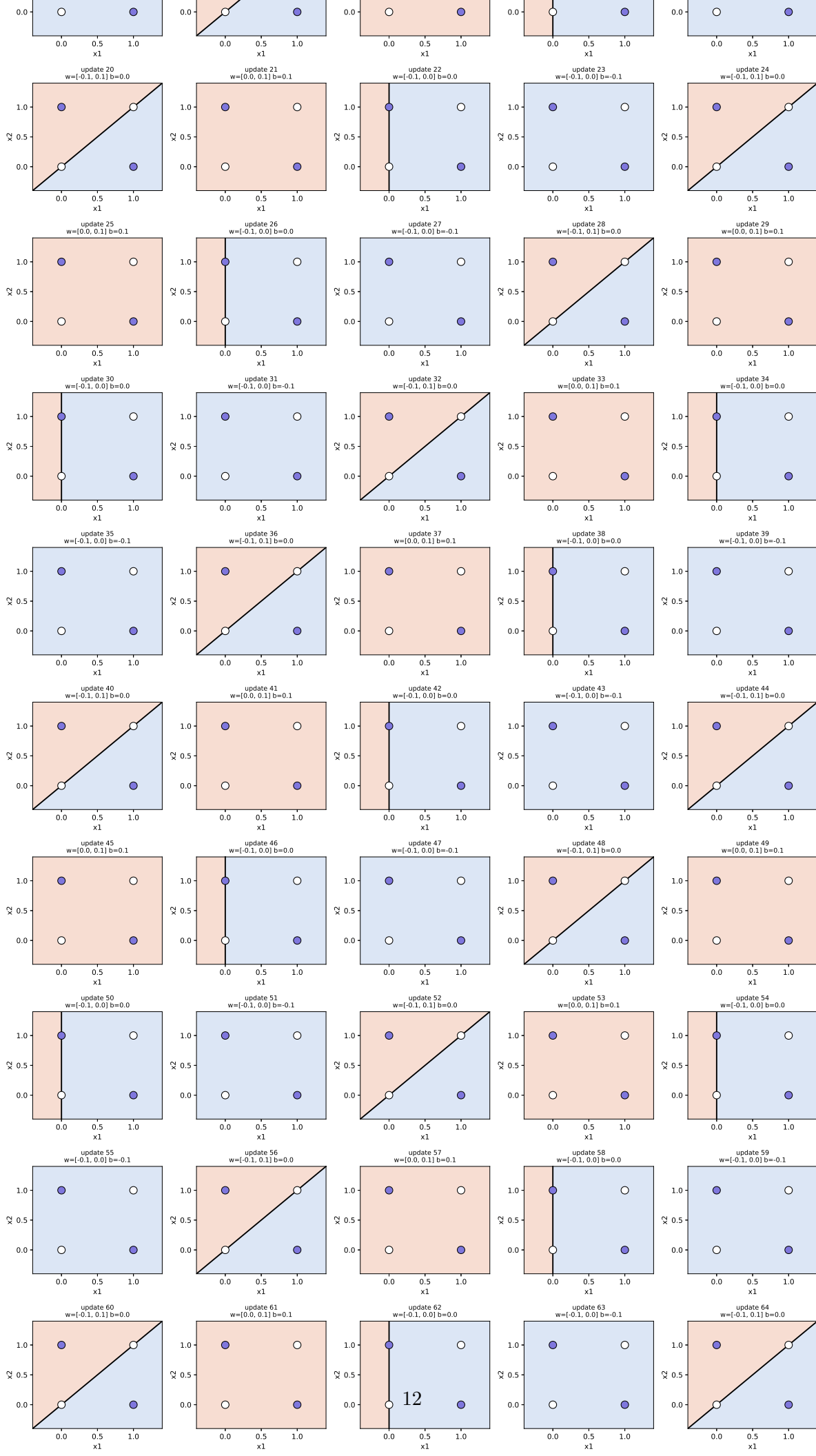


Figure 10: NOT gate: 1-D decision threshold after each update.

Inference: NOT has a single input, so its boundary is a threshold *point* on a number line, not a line in a plane. The final weight is negative ($w = -0.1$), which is the mechanism of inversion: a negative weight makes an increasing input decrease the output.



Inference: The boundary never settles. It swings back and forth between the same positions because no single straight line can separate XOR’s two positive corners (on one diagonal) from its two negative corners (on the other). The panels repeat, revealing a fixed cycle rather than progress toward a solution.

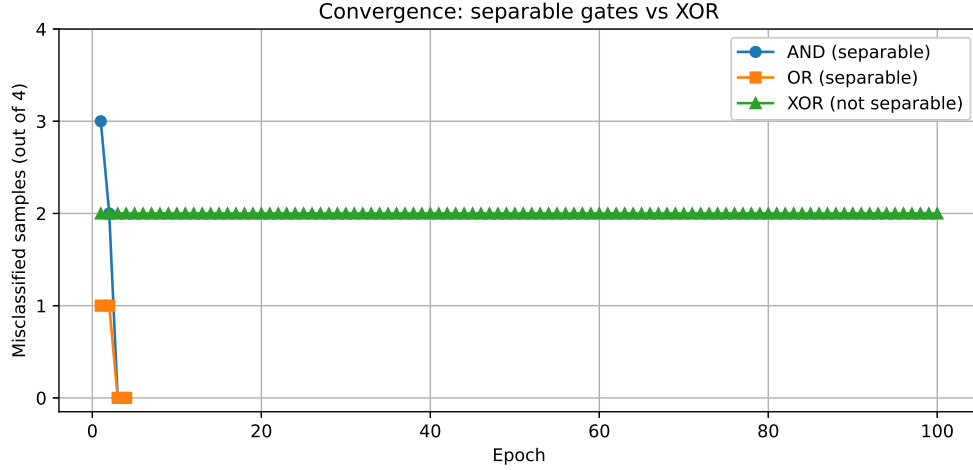


Figure 12: Misclassifications per epoch: separable gates vs XOR.

Inference: AND and OR fall to zero errors within a few epochs and stop. XOR flat-lines at 2 misclassifications out of 4 for all 100 epochs — exactly 50% — and never improves. This is the clearest single indicator of non-convergence.

XOR Analysis: The Pattern That Prevents Convergence

Three independent lines of evidence establish *why* XOR fails.

1. The weights enter a fixed cycle. Logging the weight state $(\mathbf{w}, b)$ after every update shows that only **7 distinct states** are ever visited across all 398 updates, and the sequence repeats with a **period of exactly 4** (one full epoch). Starting each epoch from $\mathbf{w} = [-0.1, 0.0]$, $b = 0.0$:

Sample	z	$\hat{y}$	y	New state $(\mathbf{w}, b)$
(0,0)	0.0	1	0	$[-0.1, 0.0], -0.1$
(0,1)	-0.1	0	1	$[-0.1, 0.1], 0.0$
(1,0)	-0.1	0	1	$[0.0, 0.1], 0.1$
(1,1)	0.2	1	0	$[-0.1, 0.0], 0.0$ (back to start)

All four samples are misclassified, each triggers an update, and the four corrections sum to the zero vector. The perceptron returns to its starting weights after every epoch and repeats identically forever — it orbits rather than converges.

2. No weights can solve XOR (algebraic proof). For the perceptron to be correct on all four inputs, the weights must satisfy:

$$\underbrace{b < 0}_{(0,0) \rightarrow 0}, \quad \underbrace{w_2 + b \geq 0}_{(0,1) \rightarrow 1}, \quad \underbrace{w_1 + b \geq 0}_{(1,0) \rightarrow 1}, \quad \underbrace{w_1 + w_2 + b < 0}_{(1,1) \rightarrow 0}.$$

Adding the two middle inequalities gives $w_1 + w_2 \geq -2b$. The fourth gives $w_1 + w_2 < -b$. Together these require

$$-2b \leq w_1 + w_2 < -b.$$

But $b < 0$ implies $-2b > -b$, so this asks $w_1 + w_2$ to be at least the larger bound yet strictly less than the smaller one — an empty interval. **No** (w_1, w_2, b) **exists**. The algorithm cannot converge because there is nothing to converge to.

3. Exhaustive search confirms it. A brute-force scan of over 10^6 weight triples on a fine grid finds **zero** that reproduce XOR, while the same scan finds thousands of valid solutions for AND. This confirms the algebraic result numerically.

Root cause. XOR requires input x_1 to *increase* the output when $x_2 = 0$ but *decrease* it when $x_2 = 1$. A single weight w_1 has one fixed sign and cannot do both. XOR needs the inputs to *interact*, and a linear model has no term for interaction. This is the historical Minsky–Papert result (1969) that motivated multi-layer networks.

Conclusion

The perceptron learning algorithm converged on AND, OR and NOT — all linearly separable — reaching zero error in a handful of epochs, with the weight traces and rotating boundaries matching theory exactly. On XOR it never converged: the weights cycle with period 4, error stays fixed at 50%, and both an algebraic proof and an exhaustive search confirm no separating line exists. The resolution is a hidden layer: a multi-layer perceptron with two hidden units solves XOR by composing two lines into a non-linear region — the same principle underlying the MLP built in the main experiment.

13 References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
4. H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms,” 2017.
5. TensorFlow / Keras Documentation, <https://keras.io>.
6. SciKeras Documentation, <https://adriangb.com/scikeras/>.